



Liberdus Token & Bridge Token Security Audit Report

Prepared by:

Arcadia

Prepared for:

Liberdus

Date: April 27th, 2026

<https://arcadia.agency/>

Email: audits@arcdiamgroup.com

Telegram: <https://t.me/thearcadiagroup>

Auditor Information

Company: Arcadia

Address: 955 W John Carpenter Frwy, Ste. 100, Irving, Texas 75039

Website: <https://arcadia.agency/>

Email: audits@arcdiamgroup.com

About Us: Arcadia is a software security and product development company building and securing software.

Client Information

Company Name: Liberdus

Website: <https://liberdus.com/>

Project Name: Liberdus Token & Bridge Contracts

Revision History

Date	Reason	Commit
04/20/2026	Initial Audit Scope	#6f5282ae / #49c83c9f
04/24/2026	Initial Delivery of Audit Report	
04/27/2026	Client Acknowledgement of Findings; Final Report Delivered	

Contents

1	Introduction	5
1.1	Audit Scope	5
1.1.1	Liberdus Token Contract (Polygon)	5
1.1.2	Liberdus Bridge Token Contract (BNB Smart Chain)	5
1.2	Project Background	6
2	On-Chain Configuration	7
2.1	Polygon: Liberdus Token	7
2.2	BNB Smart Chain: Liberdus Bridge Token	7
2.3	Shared Signer Observation	7
3	Report Summary	9
4	Audit Findings	9
4.1	[LIB-01] Bridge-in lacks transaction ID replay protection on Polygon contract	9
4.2	[LIB-02] Global bridge-in cooldown serializes all users	11
4.3	[LIB-03] Operation ID generation omits contract address on Polygon contract	12
4.4	[LIB-04] Redundant <code>whenNotPaused</code> modifier in ERC-20 transfer overrides	13
4.5	[LIB-05] Test suite asserts incorrect token symbol	14
5	Contact Information	15
6	Disclaimer	15

1 Introduction

Liberdus engaged Arcadia to perform a security audit of the Liberdus token smart contracts, covering on-chain configuration and source code across two EVM deployments. This audit is scoped to the deployed smart contracts only; off-chain bridge relayer infrastructure and the native Liberdus network are explicitly out of scope.

- **Liberdus Token (Polygon):** the primary ERC-20 token contract at <https://polygonscan.com/token/0x693ed886545970F0a3ADf8C59af5cCdb6dDF0a76>. Source at <https://github.com/Liberdus/liberdus-token-contract>.
- **Liberdus Bridge Token (BNB Smart Chain):** a secondary mint/burn ERC-20 token contract at <https://bscscan.com/token/0x5afdcc93d794762c785ec14fb2a> representing LIB on BSC. Source at <https://github.com/Liberdus/liberdus-bridge-contract>.

1.1 Audit Scope

1.1.1 *Liberdus Token Contract (Polygon)*

The primary audit target is `Liberdus.sol` deployed on Polygon Mainnet (commit `#6f5282ae`). The review covered the full contract logic, governance mechanism, bridge interface, and test suite. Components examined include:

- **Multi-signature governance:** 3-of-4 signer approval system using on-chain ECDSA signature validation, operation deadlines, and atomic execution upon threshold.
- **Token lifecycle controls:** minting to the contract address with interval enforcement and hard supply cap (`MAX_SUPPLY = 210,000,000 LIB`), burning from contract balance, and pre-launch / post-launch state machine.
- **Bridge interface:** `bridgeOut` (burn-and-emit) and `bridgeIn` (mint-on-receipt), controlled by an authorized `bridgeInCaller` address with configurable amount limits and cooldown.
- **Pause mechanism:** inherited from `OpenZeppelin Pausable`, gated behind the multisig.
- **Token distribution:** multisig-controlled transfer from contract balance to recipient addresses.
- **Test suite:** `test/liberdus.test.js` reviewed for coverage gaps and correctness.

1.1.2 *Liberdus Bridge Token Contract (BNB Smart Chain)*

The secondary audit target is `LiberdusSecondary.sol` deployed on BNB Smart Chain Mainnet (commit `#49c83c9f`). All supply enters exclusively

through `bridgeIn` calls from the authorized relayer. Components examined include:

- **Multi-signature governance:** equivalent 3-of-4 mechanism with cross-contract `operationId` isolation (`address(this)` in `operationId` hash), operation pruning, and explicit existence check for unknown operation IDs.
- **Bridge controls:** `bridgeIn` with transaction ID replay protection (`processedTxIds`), global cooldown, and per-call limit; `bridgeOut` with configurable enable flag and minimum amount enforcement.
- **Operation management:** append-only `operationId` registry with batch pruning (`pruneOperations`, `pruneOperationsByIds`) using a swap-and-pop pattern.
- **Bridge state toggles:** multisig-controlled `bridgeInEnabled` and `bridgeOutEnabled` flags.
- **Test suite:** `test/liberdus.secondary.test.js` reviewed for coverage and correctness.

1.2 Project Background

Liberdus is the native token of the Liberdus network. The token exists across three chains: the native Liberdus network, Polygon (primary EVM deployment for pre-launch distribution and governance), and BNB Smart Chain (secondary bridge deployment enabling BSC ecosystem participation). The Polygon contract serves as the canonical EVM token with full governance capabilities, while the BSC contract is a lean mint/burn bridge wrapper. A separate vault contract on Polygon handles the pre-mainnet Polygon-to-BSC locking mechanism and was not in scope for this audit.

2 On-Chain Configuration

Both contracts are verified on their respective block explorers and the verified sources were confirmed to be byte-for-byte identical to the audited GitHub repositories.

2.1 Polygon: Liberdus Token

Address: 0x693ed886545970F0a3ADf8C59af5cCdb6dDF0a76

Compiler: v0.8.20+commit.a1b79de6, no optimizer, EVM Paris

Proxy: None

Chain ID (constructor): 137 (Polygon Mainnet)

Signers:

- 0xb5a5bd462a0a76c02990d0fbe3321e92e0b03abc
- 0x1f6708632dc5720537124bc2be1eb6700384893b
- 0x6eea3d348f44a59142cdd46336bb70051c2e4f19
- 0xec33adc8a175dcc44f809909b9aae9f4f5760818

2.2 BNB Smart Chain: Liberdus Bridge Token

Address: 0x5afdcc93d794762c785ec14fb2a24c4abdbb8aaa

Compiler: v0.8.20+commit.a1b79de6, no optimizer, EVM Paris

Proxy: None

Chain ID (constructor): 56 (BNB Smart Chain Mainnet)

Current Supply: 8,203,358.82 LIB (42 holders) (as of April 20th, 2026)

Signers:

- 0x63bc72b99d3b5bc1a97bd6e843b669b65a690a66
- 0x65870f323823b654da0422beddff4e3fb2b82361
- 0xb5a5bd462a0a76c02990d0fbe3321e92e0b03abc (*shared with Polygon*)
- 0xec33adc8a175dcc44f809909b9aae9f4f5760818 (*shared with Polygon*)

2.3 Shared Signer Observation

Two of the four signers appear in both multisig configurations: 0xb5a5bd...03abc and 0xec33ad...0818. Because both multisigs require 3-of-4 approval, a party controlling these two shared keys plus any single BSC-specific signer would

reach threshold on the BSC contract simultaneously. This is not a smart contract vulnerability; it is an operational key-management consideration for the Liberdus team to be aware of when assessing their overall key compromise blast radius.

3 Report Summary

This audit identified a total of **5 findings** across the two audited contracts. Following review, the Liberdus team has responded to all five findings: two are accepted as known and manageable risks, and three are reported by the team as fixed in newer versions of the contracts. The findings are categorized by severity as follows:

Severity Level	Count	Acknowledged	Reported Fixed	Finding IDs
Medium	1	0	1	LIB-01
Low	1	1	0	LIB-02
Informational	3	1	2	LIB-03, LIB-04, LIB-05

Client Response. On April 27th, 2026, the Liberdus team formally responded to this audit. The team accepted LIB-02 and LIB-04 as known and manageable risks, and reported that LIB-01, LIB-03 and LIB-05 have been addressed in newer versions of the contracts. For LIB-01, the team additionally states that safeguards exist on the tss-node side of the bridge.

Scope of Verification. The remediation statuses recorded in this report reflect statements made by the Liberdus team. The newer contract versions referenced by those statements were not provided to Arcadia and have not been audited, reviewed or verified by Arcadia. Likewise, the off-chain tss-node safeguards cited in relation to LIB-01 were not provided for review and fall outside the scope of this engagement, which covers only the deployed contracts listed in Section 2. The deployed contracts at the audited addresses remain as described in this report. Arcadia makes no assurance as to the correctness or completeness of any fix in the newer versions, and recommends that those versions receive their own audit before deployment.

4 Audit Findings

4.1 [LIB-01] Bridge-in lacks transaction ID replay protection on Polygon contract

Risk Level

Severity: Medium, Likelihood: Low, Status: Reported Fixed (Unverified)

Code Location

- `contracts/Liberdus.sol: bridgeIn(address,uint256,uint256,bytes32)`

Description

The `bridgeIn` function on the Polygon contract mints tokens to a recipient when called by the authorized `bridgeInCaller`. It enforces a global cooldown

and a per-call amount cap, but it does *not* record or check the supplied `txId` parameter. This means the same source-chain transaction ID can be submitted to the contract any number of times, and each submission will successfully mint tokens as long as the cooldown has elapsed.

```
1 function bridgeIn(address to, uint256 amount, uint256 _chainId,
2   bytes32 txId)
3   public onlyBridgeInCaller whenNotPaused
4 {
5   require(!isPreLaunch, "Bridge in not available in pre-launch");
6   require(_chainId == chainId, "Invalid chain ID");
7   require(amount > 0, "Cannot bridge in zero tokens");
8   require(amount <= maxBridgeInAmount, "Amount exceeds bridge-in
9     limit");
10  require(block.timestamp >= lastBridgeInTime + bridgeInCooldown,
11    "Bridge-in cooldown not met");
12
13  _mint(to, amount);
14  lastBridgeInTime = block.timestamp;
15  emit BridgedIn(to, amount, _chainId, txId, block.timestamp);
16 }
```

Listing 1: Liberdus.sol – bridgeIn, no txId check

By contrast, the BNB bridge contract (`LiberdusSecondary.sol`) correctly prevents this with an explicit replay guard:

```
1 mapping(bytes32 => bool) public processedTxIds;
2
3 require(!processedTxIds[txId], "Transaction already processed");
4 // ...
5 processedTxIds[txId] = true;
```

Listing 2: LiberdusSecondary.sol – bridgeIn with replay guard

Impact

A compromised or malicious `bridgeInCaller` can replay any historical `txId` once per cooldown period, minting tokens without a corresponding burn on the source chain. Given the cooldown is 1 minute by default and the per-call cap is 10,000 LIB, an attacker controlling the key could inflate supply at up to 10,000 LIB/minute with no on-chain bound other than the hard cap. Even under normal operations, a software bug in the relayer could accidentally double-mint for the same source event.

Recommendation

Add a `processedTxIds` mapping and check-then-set it in `bridgeIn`, mirroring the pattern already implemented in `LiberdusSecondary.sol`:

```
1 mapping(bytes32 => bool) public processedTxIds;
```

```
2
3 function bridgeIn(..., bytes32 txId) public onlyBridgeInCaller
4     whenNotPaused {
5         // existing checks ...
6         require(!processedTxIds[txId], "Transaction already processed");
7         _mint(to, amount);
8         processedTxIds[txId] = true;
9         lastBridgeInTime = block.timestamp;
10        emit BridgedIn(to, amount, _chainId, txId, block.timestamp);
11    }
```

Remediation

The Liberdus team reports that this issue is handled in a newer version of the contract, and states that additional safeguards against replayed bridge-in requests exist on the tss-node side of the bridge relayer. Arcadia was not provided with either the newer contract version or the tss-node implementation, and has therefore not verified the on-chain fix or the claimed off-chain safeguards. Both fall outside the scope of this engagement. The deployed Polygon contract at 0x693ed8...0a76 remains without on-chain txId replay protection as described above, so the risk for that deployment rests entirely on controls Arcadia has not reviewed.

4.2 [LIB-02] Global bridge-in cooldown serializes all users

Risk Level

Severity: Low, Likelihood: Medium, Status: Acknowledged (Known Issue)

Code Location

- contracts/Liberdus.sol: bridgeIn(), lastBridgeInTime
- contracts/LiberdusSecondary.sol: bridgeIn(), lastBridgeInTime

Description

Both contracts maintain a single global lastBridgeInTime timestamp. Only one bridgeIn call can succeed per cooldown window (default: 1 minute) across all users and all pending bridge transactions. If two users initiate bridge-in transfers that arrive at the relay within the same minute, one will revert with “Bridge-in cooldown not met” and require a manual retry.

```
1 uint256 public lastBridgeInTime;
2 uint256 public bridgeInCooldown = 1 minutes;
3
4 require(block.timestamp >= lastBridgeInTime + bridgeInCooldown,
5         "Bridge-in cooldown not met");
6 // ...
```

```
7 | lastBridgeInTime = block.timestamp;
```

Listing 3: Global cooldown state

Impact

Under light usage the impact is negligible, as concurrent bridge-in transactions are unlikely. However, as the protocol grows, simultaneous bridge-ins become more probable, causing intermittent relay failures and a degraded user experience. A sufficiently active attacker could also deliberately front-run bridge-in calls to force legitimate users into repeated cooldown delays, though the economic benefit to the attacker is limited.

Recommendation

Consider one of the following mitigations depending on operational requirements:

- **Per-transaction replay protection only:** remove the cooldown entirely and rely solely on the `processedTxIds` check (already present in `LiberdusSecondary`; recommended as part of LIB-01 remediation for the Polygon contract). Replay protection alone is sufficient to prevent double-minting.
- **Retain cooldown with higher limits:** if the cooldown serves a rate-limiting purpose, increase `maxBridgeInAmount` to cover the expected burst volume within a single cooldown window, reducing the chance of legitimate transactions colliding.

Remediation

The Liberdus team has acknowledged this as a known issue and accepted the associated risk. The global cooldown remains in place on both deployed contracts, and the serialization behavior described above should be expected to persist as bridge volume grows.

4.3 [LIB-03] Operation ID generation omits contract address on Polygon contract

Risk Level

Severity: Informational, Likelihood: N/A, Status: Reported Fixed (Unverified)

Code Location

- `contracts/Liberdus.sol: requestOperation()`

Description

The Polygon contract computes `operationId` as:

```
1 bytes32 operationId = keccak256(abi.encodePacked(  
2     operationCount++, opType, target, value, data, chainId  
3 ));
```

The contract address (`address(this)`) is not included. If a second instance of `Liberdus.sol` were deployed on the same chain with the same `chainId` value, the two contracts would produce identical `operationId` hashes for the same sequence of operations, making signatures portable between them. The BNB bridge contract corrected this in a later commit by adding `address(this)` to the preimage.

Impact

The practical risk is low: the Polygon deployment is a singleton, and the `chainId` parameter provides chain-level domain separation. The concern is confined to a future redeployment or migration scenario where a second instance shares the same chain and `chainId`.

Recommendation

Add `address(this)` to the `operationId` hash preimage, consistent with `LiberdusSecondary.sol`.

```
1 bytes32 operationId = keccak256(abi.encodePacked(  
2     operationCount++, opType, target, value, data, chainId,  
3     address(this)  
4 ));
```

Remediation

The Liberdus team reports that this issue is fixed in a newer version of the contract. That version was not provided to Arcadia and has not been reviewed, so the fix is unverified. The deployed Polygon contract still omits `address(this)` from the `operationId` preimage.

4.4 [LIB-04] Redundant `whenNotPaused` modifier in ERC-20 transfer overrides

Risk Level

Severity: Informational, Likelihood: N/A, Status: Acknowledged

Code Location

- `contracts/Liberdus.sol: transfer(), transferFrom(), _update()`

Description

The contract overrides `transfer` and `transferFrom` with the `whenNotPaused` modifier, and also overrides `_update` (the internal hook called by both func-

tions) with the same modifier. Because all ERC-20 token movements flow through `_update`, the pause check runs twice on every transfer.

```
1 function transfer(address to, uint256 amount)
2     public override whenNotPaused returns (bool) { ... }
3
4 function transferFrom(address from, address to, uint256 amount)
5     public override whenNotPaused returns (bool) { ... }
6
7 function _update(address from, address to, uint256 amount)
8     internal override whenNotPaused { ... }
```

Impact

Functionally correct. The redundancy wastes a small amount of gas on every transfer, mint, and burn.

Recommendation

Remove `whenNotPaused` from `transfer` and `transferFrom` and retain it only in `_update`, which is the canonical interception point for all token movements in OpenZeppelin ERC-20 v5.

Remediation

The Liberdus team has acknowledged this finding and accepted the additional gas cost. No change has been made to the deployed contract.

4.5 [LIB-05] Test suite asserts incorrect token symbol

Risk Level

Severity: Informational, Likelihood: N/A, Status: Reported Fixed (Unverified)

Code Location

- `test/liberdus.test.js`: line 51
- `contracts/Liberdus.sol`: constructor

Description

The test suite asserts `symbol() == "LBD"`, but the contract is initialized as `ERC20("Liberdus", "LIB")`. The on-chain symbol is "LIB", as confirmed by the verified source on Polygonscan. This test fails against the current code-base, indicating the suite was not run before the most recent changes were committed.

Impact

No impact on production contract behavior. Failing tests erode confidence in the suite and may mask future regressions if CI is not enforced.

Recommendation

Update the assertion to "LIB" and enforce that all tests pass in CI before merging changes to the main branch.

Remediation

The Liberdus team reports that the assertion is corrected in a newer version of the test suite. That version was not provided to Arcadia and has not been reviewed, so the fix is unverified.

5 Contact Information

For further inquiries or to schedule a follow-up audit, please contact:

Arcadia

Email: audits@arcadiamgroup.com

Website: <https://arcadia.agency/>

Telegram: <https://t.me/thearcadiagroup>

6 Disclaimer

While best efforts and precautions have been taken in preparing this document, Arcadia and the Authors assume no responsibility for errors, omissions, or damages resulting from the use of the provided information. Additionally, Arcadia would like to emphasize that the use of Arcadia's services does not guarantee the security of a smart contract or set of smart contracts and does not guarantee against attacks. One audit on its own is not enough for a project to be considered secure; that categorization can only be earned through extensive peer review and battle testing over an extended period.